

Project for the course Efficient Programs WS24/25

You can choose an arbitrary project.

If you don't have any project of your own, you can choose the [given project](#) for this semester.

Implementing the given project has some advantages: You don't need to spend precious presentation time on explaining the problem, and may also be able to spend less on explaining the algorithm, and the results are directly comparable to those of other groups. A disadvantage of using the given project is that you might be repeating what others have done and you may produce worse results (especially if you choose a later presentation date). If your results are worse, this does not influence the grading, though; the grading is based on whether you demonstrate that you have applied the content of the course.

Prepare an 18-minute presentation (I recommend doing a trial run). You do not have as much time as I had in the lecture, so I recommend that you present most optimization steps only superficially, and focus on those steps in more detail that are particularly interesting, e.g., because they produced surprisingly good or surprisingly bad results.

Your presentation should demonstrate that you have applied the lessons taught in the course, e.g., concentrating on the parts that consume most time.

Given Project: myjoin

Implement a [join](#) of four files, with the first, second, and third file joining with their first field, and the second field of the third file joining with the first field of the fourth file, with the output records consisting of, in this order: the first field of file4, the first field of file1, the second field of file1, the second field of file2, and the second field of file4. This is expressed in the reference implementation [myjoin](#), which is invoked on the g0 as follows:

```
/usr/ftp/pub/anton/lvas/effizienz-aufgabe24/myjoin file1 file2 file3 file4
```

The records of both input files and the output (on the standard output) are separated by newlines and the fields are separated by commas.

If you are familiar with Prolog or Datalog, myjoin corresponds to having a set of facts for each file, and a predicate myjoin/5 defined as:

```
myjoin(D,A,B,C,E) :-
    file1(A,B),
    file2(A,C),
    file3(A,D),
    file4(D,E).
```

The output of myjoin is the multiset of solutions for this predicate (each solution is an output record); e.g., in Prolog this output can be produced with:

```
?- myjoin(D,A,B,C,E), print((D,A,B,C,E)), nl, fail.
```

Note that, if a value occurs n times as the join field of one file and m times as the join field of another file, this results in $n*m$ output records from these two files alone (further multiplications arise from joining with the other two files); this is also true if a value occurs 0 times in the join field of one of the files.

The output of your implementation must contain the same records as the output of the reference

implementation, but they can be output in a different order than produced by `myjoin`. You can compare with the output of the given program `myjoin` by performing (on `g0`)

```
#small data set
cd /localtmp/efficient24
yourjoin a.csv b.csv c.csv d.csv|sort|diff - abcd.csv
#big data set
cd /localtmp/efficient24
yourjoin f1.csv f2.csv f3.csv f4.csv|sort|diff - output.csv
```

An empty output means that your output is correct.

I recommend doing the test runs on the `g0`, but if you want to perform them elsewhere, you can download the data: [data.tar.xz](#) (801MB)

To report your optimizations, use

```
cd /localtmp/efficient24
LC_NUMERIC=en_US perf stat -e cycles yourjoin f1.csv f2.csv f3.csv f4.csv|cat >/dev/null
```

The reference implementation takes 155,414,886,890 cycles (distributed across several processes). Please report the result of your implementation by running it on the `g0` with the same command (replace "`yourjoin`").

Note that the reference implementation has a significantly shorter elapsed time (~11s) than CPU time (~39s) due to pipeline parallelism; for the given task we are mainly interested in the CPU time, so if you write a single-threaded implementation that takes more elapsed time, this can still be a win with respect to our evaluation criterion.

For understanding the performance, you can also measure other events, such as `-e instructions -e branches -e branch-misses -e L1-dcache-load-misses`, or follow the top-down microarchitectural analysis approach discussed in the lecture part. For this project the amount of cycles (or other events) spent in system calls may also be significant, and you can check for that by appending `:k` to the event name. Conversely, append `:u` for events spent in user mode.

The course is not about algorithms, but that does not mean that you have to use the algorithm used by the reference implementation, and in fact I would not recommend that. Feel free to implement `myjoin` with whatever algorithm and data structure you find promising, and then optimize the resulting program.

Putting the results online

You can publish your program and its documentation (e.g., the presentation) by putting a project on, e.g., Github or [Sourcehut](#), and putting a web page with a link to that project on `/nfs/unsafe/httpd/ftp/pub/anton/lvas/effizienz-abgaben/2024w` (on the `g0`). Alternatively, you can also create a directory there and put your program and documentation there. This web page or directory is visible in the web through [this page](#).

The publication of your solution is not graded, so publishing it is completely optional.

Project tasks from [[WS02/03](#) | [WS03/04](#) | [WS04/05](#) | [WS05/06](#) | [WS06/07](#) | [WS07/08](#) | [WS08/09](#) | [WS09/10](#) | [WS10/11](#) | [WS11/12](#) | [WS12/13](#) | [WS13/14](#) | [WS14/15](#) | [WS15/16](#) | [WS16/17](#) | [WS17/18](#) | [WS18/19](#) | [WS19/20](#) | [WS20/21](#) | [WS21/22](#) | [WS22/23](#) | [WS23/24](#)]

[Anton Ertl](#)

Name	Last modified	Size	Description
----------------------	-------------------------------	----------------------	-----------------------------

 Parent Directory	-
 data.tar.xz	20-Nov-2024 13:40 800M
 generator/	18-Nov-2024 13:21 -
 myjoin	02-Nov-2024 18:16 249
 myjoin.pl	18-Nov-2024 14:50 282

*Apache/2.2.22 (Debian) DAV/2 mod_fcgid/2.3.6 PHP/5.4.36-0+deb7u3 mod_python/3.3.1 Python/2.7.3
mod_ssl/2.2.22 OpenSSL/1.0.1e mod_perl/2.0.7 Perl/v5.14.2 Server at www.complang.tuwien.ac.at Port
443*